



Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II.

doi: 10.18637/jss.v000.i00

gsDesignNB: Group Sequential Design and Sample Size Re-estimation for Negative Binomial Recurrent Event Trials

Keaven Anderson

Merck & Co., Inc., Rahway, NJ, USA

Hongtao Zhang

Merck & Co., Inc., Rahway, NJ, USA

Nan Xiao

Genentech, Inc., South San Francisco, CA, USA

Andrea Maes

Merck & Co., Inc., Rahway, NJ, USA

Abstract

We present the R package **gsDesignNB** for planning, monitoring, and adapting clinical trials with recurrent event endpoints analyzed using negative binomial models. The package links fixed design sample size calculations, calendar-time group sequential design construction, blinded and unblinded information estimation, sample size re-estimation (SSR), and recurrent-event simulation in a single workflow built on the **gsDesign** package. In the working model, conditional on treatment arm and a subject-level gamma frailty, recurrent events follow a Poisson process with constant within-subject rate, yielding marginal negative binomial counts; a seasonal-rate extension relaxes the constant-rate assumption for respiratory and other calendar-time-dependent applications. For fixed designs, the package extends the methods of [Zhu and Lakkis \(2014\)](#) and [Friede and Schmidli \(2010\)](#) to handle piecewise accrual, piecewise exponential dropout, maximum follow-up truncation, and protocol-defined minimum intervals between countable events (combining event duration and required symptom-free days). Because statistical information for the log rate ratio depends jointly on exposure, event rates, and dispersion, it is not generally proportional to raw event counts. We derive and implement a second-order delta-method correction for the Jensen-inequality bias in the event-gap effective-rate approximation; in a 64-scenario simulation sweep, the correction reduces representative approximation error from about 8% to 1% and prevents 1–2.5 percentage points of underpowering when dispersion is moderate-to-high and gaps exceed 20 days. A second methodological advance is a score-test workflow with explicit comparison to the traditional Wald/Zhu–Lakkis sample-size formula. The Wald formula uses the alternative variance for both Type I error and power, whereas the score formula uses the null variance for the Type I component and the alternative variance for the power component. A factorial null simulation shows mild finite-sample Type I error inflation for the Wald test and conservative Type I behaviour for the score test; in the displayed superiority scenarios, retaining Wald sizing while using the score test provides a small practical power margin without losing Type I protection. The package also implements a disciplined fallback from maximum likelihood to method-of-moments (MoM) nuisance-parameter estimation: when the NB ML fit is near-Poisson the software switches to a Poisson Wald or score test, but under extreme overdispersion or ML non-convergence it switches to a MoM-based NB test using the observed Fisher information formula, avoiding the Type I error inflation of a blind Poisson fallback. The package also supports calendar-time group sequential monitoring and both blinded and unblinded SSR. A 90-scenario SSR article illustrates the adaptive simulation engine, and production-scale null simulations show that SSR should be checked under the null with the planned final test statistic. In our examples, the Wald test exhibits mild Type I error inflation after SSR, whereas the score test preserves Type I error more conservatively. The paper emphasizes both the statistical methodology and the software architecture that

1. Introduction

Recurrent event endpoints are common in clinical trials for conditions such as multiple sclerosis relapses, COPD exacerbations, and heart failure hospitalizations. When every patient has the same underlying event rate, a Poisson model suffices: the variance equals the mean, and information accrues linearly with exposure. Real patient populations, however, exhibit substantial between-patient heterogeneity in event rates. Sicker patients contribute many events while most contribute few, inflating the observed variance well beyond the Poisson mean. Ignoring this overdispersion leads to inflated Type I error and overstated power, understating the sample size needed to detect a clinically meaningful treatment effect.

The negative binomial (NB) model addresses this by adding a single dispersion parameter k that captures the excess variation. Published NB fits from pivotal or large trials confirm that $k > 0$ is the rule rather than the exception: reported estimates range from approximately 0.3 to 3.0 or higher across disease areas including COPD exacerbations (Wedzicha, Banerji, Chapman, Vestbo, Roche, Ayers, Thach, Fogel, Patalano, and Vogelmeier 2016; Lipson, Barnhart, Brealey, Brooks, Criner, Day, Dransfield, Halpin, Han, Jones *et al.* 2018), heart failure hospitalizations (McMurray, Packer, Desai, Gong, Lefkowitz, Rizkala, Rouleau, Shi, Solomon, Swedberg, and Zile 2014), and multiple sclerosis relapses (Hauser, Bar-Or, Comi, Giovannoni, Hartung, Hemmer, Lublin, Montalban, Rammohan, Selmaj *et al.* 2017). As a rough calibration, $k < 0.3$ indicates near-Poisson behaviour, $0.3 \leq k \leq 1.0$ is moderate overdispersion (typical of many pivotal trials), $k > 1.5$ is high (common for severe COPD exacerbations and asthma endpoints), and $k > 5$ is extreme (where method-of-moments fallback estimation may become relevant). Dispersion is rarely reported directly in publications; it can often be back-calculated from published mean and variance (or confidence intervals) of event counts, or from the NB shape parameter $\theta_{\text{NB}} = 1/k$ in `MASS::glm.nb()` notation. The package function `estimate_nb_mom()` supports method-of-moments estimation from summary statistics. Even moderate dispersion materially reduces the Fisher information for the log rate that each subject contributes — from μ_i under the Poisson to $\mu_i/(1 + k\mu_i)$ under the NB — so that a design ignoring k can understate the required sample size by 10–30%.

Negative binomial design work also becomes operationally more complicated once a trial moves beyond a fixed final analysis to include interim analyses. Information for the log rate ratio depends jointly on exposure, event rates, and dispersion, so raw event counts are only an incomplete proxy for monitoring. Accrual may be piecewise, dropout may truncate follow-up, interim looks may be scheduled on the calendar rather than exactly on information fractions, and some protocols only count events separated by a minimum patient-level gap. In software, these trial features must be represented consistently in planning formulas, recurrent-event simulation, interim data cuts, and alpha-spending calculations.

The **gsDesignNB** package for R provides an integrated toolkit for these tasks:

1. Sample size and power for fixed designs with flexible enrollment, dropout, follow-up truncation, protocol event gaps, and either Wald or score-test sizing.
2. Calendar-time group sequential designs that inherit spending-function infrastructure from the **gsDesign** package (Jennison and Turnbull 1999).
3. Blinded and unblinded information estimation together with blinded and unblinded SSR (Friede and Schmidli 2010; Mütze, Glimm, Schmidli, and Friede 2019), with a disciplined maximum likelihood / method-of-moments fallback scheme that keeps Wald and score

inference well-defined under extreme overdispersion or ML non-convergence.

4. Recurrent-event simulation, including seasonal rates, event- and information-driven interim cutting rules, and operating-characteristic summaries.

The contribution of **gsDesignNB** is therefore both scientific and computational. Scientifically, the package implements NB design formulas for settings where information is not proportional to events, introduces a Jensen-correction for event-gap planning, adds score-test sizing for designs where null calibration is paramount, and replaces the usual blind Poisson fallback with a method-of-moments NB test under genuine overdispersion. Computationally, it connects planning objects, calendar-time monitoring rules, subject-level simulated trial data, and interim analysis summaries in one workflow. The remainder of the paper first describes that workflow, including the different notions of “gaps” that arise in recurrent-event trials, and then presents the methodological and simulation results that motivated the software design.

2. Package overview and workflow

2.1. Scope and relation to gsDesign

`sample_size_nbinom()` is the fixed-design entry point. It returns a `sample_size_nbinom_result` object containing the planning assumptions, expected exposure, expected events, and variance of the log rate ratio. `gsNBCalendar()` lifts that object into a `gsNB` design that inherits from `gsDesign`, so the NB-specific information calculations live in **gsDesignNB** while spending functions, boundary summaries, and integer rounding remain compatible with the underlying **gsDesign** ecosystem. For simulation, `nb_sim()` and `nb_sim_seasonal()` generate subject-level recurrent-event histories, `cut_data_by_date()` and related helpers convert them to interim analysis data sets, and `sim_gs_nbinom()` or `sim_ssr_nbinom()` produce operating characteristics that can be summarized with `summarize_gs_sim()` and `summarize_ssr_sim()`.

Task	Primary functions	Main outputs
Planning		
Fixed design	<code>sample_size_nbinom()</code> , <code>compute_info_at_time()</code>	<code>sample_size_nbinom_result</code> with Wald or score-test sizing
Calendar-time GS design	<code>gsNBCalendar()</code> , <code>toInteger()</code> , <code>gsBoundSummary()</code>	<code>gsNB</code> object with information and calendar timing
Data generation and interim construction		
Recurrent-event data generation	<code>nb_sim()</code> , <code>nb_sim_seasonal()</code>	Subject-level event histories

Task	Primary functions	Main outputs
Interim data construction	<code>cut_data_by_date()</code> , <code>get_cut_date()</code> , <code>get_analysis_date()</code> , <code>cut_completers()</code>	One row per randomized subject with events and exposure
Inference and monitoring		
Interim inference and information	<code>mutze_test()</code> , <code>calculate_blinded_info()</code> , <code>estimate_nb_mom()</code>	Wald or score statistics and blinded/unblinded information estimates
GS operating-characteristic simulation	<code>sim_gs_nbinom()</code> , <code>check_gs_bound()</code> , <code>summarize_gs_sim()</code>	Boundary-crossing summaries
Adaptive designs		
Adaptive simulation	<code>blinded_ssr()</code> , <code>unblinded_ssr()</code> , <code>sim_ssr_nbinom()</code> , <code>summarize_ssr_sim()</code>	Adapted sample sizes and SSR summaries

The package also re-exports the common **gsDesign** spending functions so that users can stay in one namespace when they move from fixed design planning to spending-function specification. Internally, the simulation engines rely on **data.table** for aggregation and on reproducible parallel random number streams for larger Monte Carlo studies.

2.2. Worked design example

To make the workflow concrete, consider a semi-real recurrent-event superiority trial inspired by exacerbation studies. The control-arm event rate is 0.5 events/month, the target rate ratio is 0.70, the dispersion is $k = 0.5$, maximum follow-up is 12 months, annual dropout is 10%, and the endpoint counts only events separated by at least 20 days. Accrual is piecewise, with 12 participants/month for the first 6 months and 24 participants/month for the next 6 months. The planned analysis uses the score test with one-sided $\alpha = 0.025$ and 90% power. The futility boundary uses a Cauchy spending function ([Anderson and Clark 2010](#)) with parameters specifying that 56% and 63% of total futility spending occur by information fractions 0.4 and 0.75, respectively; this yields spending that increases relatively quickly between information fractions 0 and 0.2, flattens between 0.2 and 0.85, and then rises again from 0.85 to 1.

```
ex_fixed <- sample_size_nbinom(
  lambda1 = 0.5,           # control event rate (events/month)
  lambda2 = 0.35,          # experimental event rate (RR = 0.70)
  dispersion = 0.5,        # NB overdispersion parameter k
  power = 0.90,            # target power
  alpha = 0.025,           # one-sided Type I error
  accrual_rate = c(12, 24), # relative enrollment rates (ramp-up)
  accrual_duration = c(6, 6), # duration of each accrual period (months)
```

```

trial_duration = 24,          # total study duration (months)
dropout_rate = 0.10 / 12,     # monthly dropout rate (10% annual)
max_followup = 12,           # maximum per-subject follow-up (months)
event_gap = 20 / 30.4375,    # 20-day minimum gap between counted events
test_type = "score"          # score-test sizing
)

# Note: the gsDesign argument `k` below is the number of analyses (K),
# not the NB dispersion parameter k used elsewhere in this paper.
ex_gs <- ex_fixed |>
  gsNBCalendar(
    k = 3,                    # K = 3 analyses (2 interim + final)
    test.type = 4,           # non-binding futility bound
    alpha = 0.025,           # one-sided alpha
    sfu = sfHSD,              # efficacy: Hwang-Shih-DeCani
    sfupar = -2,              # HSD gamma parameter
    sfl = sfCauchy,           # futility: Cauchy spending
    sflpar = c(0.4, 0.75, 0.56, 0.63), # 56%/63% spending at IF 0.4/0.75
    analysis_times = c(9, 14, 24) # calendar months for each look
  ) |>
  gsDesignNB::toInteger()

```

Quantity	Value
Fixed-design sample size	268
Group sequential final sample size	324
Target information	99.8
Expected final counted events	1117.8
Average final exposure	11.4
Score-test sizing	score

Table 2: Summary of the semi-real recurrent-event design example.

The fixed design requires 268 participants. After calendar-time group sequential monitoring and integer rounding, the final maximum sample size is 324. The same object stores the expected information, exposure, and event totals at each analysis. These quantities are the inputs used by the simulation engines, so the operating-characteristic calculations inherit the same calendar and recurrent-event assumptions.

Look	Month	Information fraction	Cumulative N	Expected events	Upper Z	Lower Z
IA 1	9	0.323	216	239.1	2.691	0.423
IA 2	14	0.726	324	651.6	2.310	0.912
Final	24	1.000	324	1117.8	2.076	2.073

Look	Month	Information fraction	Cumulative	Expected events	Upper	Lower Z
			N		Z	

Table 3: Calendar-time group sequential looks for the design example.

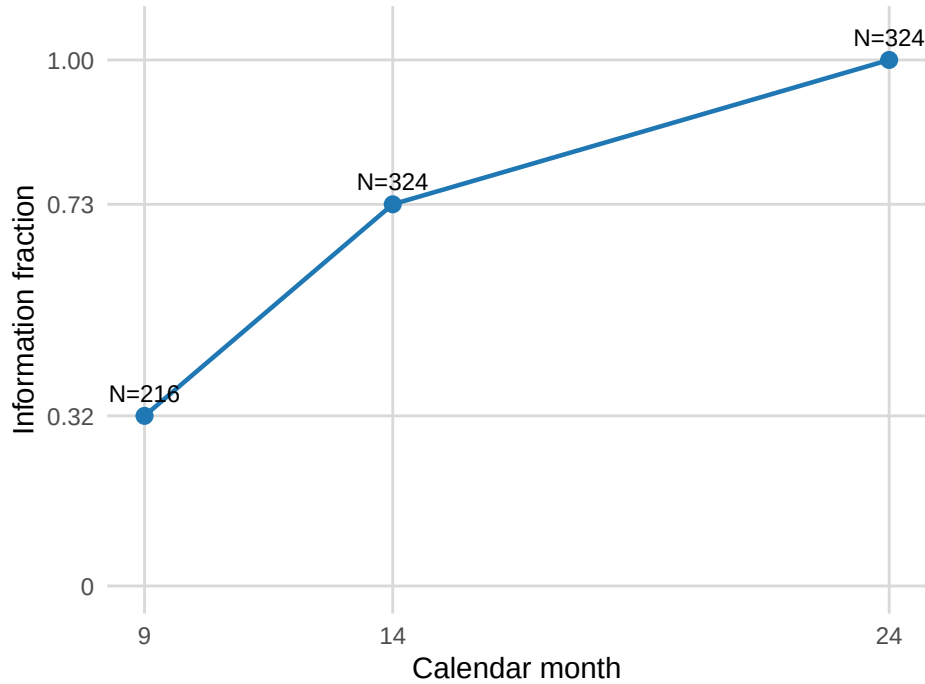


Figure 1: Information fraction and cumulative sample size for the design example.

The worked example is intentionally small enough to render as part of the manuscript, but it is not a toy calculation: it uses piecewise accrual, dropout, maximum follow-up, an event gap, score sizing, calendar-time monitoring, non-binding futility, and integer rounding. The longer vignettes reuse the same objects for simulation and SSR operating-characteristic studies.

2.3. Calendar timing, enrollment, and event gaps

Interim and final analyses are specified as calendar times (months from first enrollment) via the `analysis_times` argument to `gsNBCalendar()`. This is computationally simpler than solving for the calendar dates at which target information fractions are reached, and in practice it is straightforward to adjust calendar times to approximate desired information fractions. At the simulation stage, `sim_gs_nbinom()` and `sim_ssr_nbinom()` can combine multiple requirements (e.g., both a minimum calendar time and a minimum information fraction) to trigger each analysis.

The `accrual_rate` and `accrual_duration` arguments together describe the *relative* enrollment pattern over time. For example, `accrual_rate = c(12, 24)` with `accrual_duration =`

`c(6, 6)` specifies a 12-month enrollment period in which the rate doubles after the first 6 months. When sizing the trial, `sample_size_nbinom()` multiplies all values in `accrual_rate` by a common constant so that the total sample size achieves the targeted power; thus `accrual_rate` defines the enrollment *shape*, not the absolute number of subjects per month.

Some recurrent-event protocols only count events separated by a minimum patient-level interval. This interval typically combines two components: the duration of the event itself (e.g., a 7-day course of corticosteroids for a COPD exacerbation) plus a protocol-defined separation period to declare a subsequent event as distinct (e.g., an additional 7 days symptom-free). The `event_gap` argument represents the *total* exclusion window after each counted event; the user is responsible for summing any event-duration and separation components appropriate to their protocol. For example, a COPD trial with 7-day steroid courses and a 7-day new-event rule would use `event_gap = 14 / 30.4375` (14 days expressed in months). The `event_gap` parameter can represent either a start-to-start or an end-to-start convention, depending on what the user includes in the total: a start-to-start rule simply sets `event_gap` to the minimum start-to-start interval, whereas an end-to-start rule adds the expected event duration.

The package implements an “excluding recovery time” (ERT) model: the gap reduces both the effective event rate and the at-risk exposure. In `nb_sim()`, events are generated sequentially so that after each counted event, the next event cannot occur until at least `event_gap` time units later — this is an integral part of the data-generation process, not a post-hoc discard. The same gap rule is applied in the planning formulas (`sample_size_nbinom()`) and data-cutting functions (`cut_data_by_date()`), keeping counted events and exposure aligned throughout the workflow.

Protocols that use an “always-at-risk” (AAR) exposure offset — where the full calendar time on study enters the log-exposure term regardless of event-gap periods — should set `event_gap = 0` and combine events externally according to their protocol-defined counting rule. Under AAR, event gaps affect event *counting* but not the exposure offset, so the gap-correction machinery in the package is not needed.

2.4. Primary and secondary workflows

For most users, the package workflow is:

1. Use `sample_size_nbinom()` to build a fixed design under the planned rates, dispersion, accrual, dropout, follow-up assumptions, and planned final test statistic.
2. Convert that result to a group sequential design with `gsNBCalendar()`, optionally rounding with `toInteger()`.
3. Choose the appropriate simulator: `sim_gs_nbinom()` for calendar-time GS verification or `sim_ssr_nbinom()` for adaptive simulations with blinded or unblinded SSR.
4. Summarize operating characteristics with `check_gs_bound()`, `summarize_gs_sim()`, and `summarize_ssr_sim()`.

The package also includes several secondary workflows that are important in practice but do not change the core architecture: seasonal recurrent-event simulation, completer-driven interim looks, non-inferiority and super-superiority designs, blinded-information diagnostics, and an interactive Shiny front end to the SSR engine. These are enumerated with their corresponding vignettes in Section 6.3.

2.5. Dropout and missing data

Dropout is represented in **gsDesignNB** as exposure truncation. At the planning stage, `sample_size_nbinom()` accounts for piecewise exponential dropout and maximum follow-up through the expected exposure $\bar{t}_g$ in each treatment arm; at interim analyses, data-cutting functions retain each subject's observed counted events and observed exposure time, including partial follow-up before dropout or the analysis cut. Every randomized subject contributes to the analysis regardless of whether they complete follow-up — including subjects who discontinue before experiencing any event (contributing observed exposure > 0 and event count $= 0$). Each subject's counted events and observed exposure time enter the negative binomial likelihood through the log-exposure offset, making this an observed-exposure likelihood analysis rather than a complete-case analysis that discards early discontinuers.

Under an ignorable missing-data process — missing at random (MAR) together with distinct parameters for the observation and outcome processes — direct likelihood inference is valid without modelling the missingness mechanism (Rubin 1976). This includes the common treatment-policy estimand scenario in which subjects who discontinue treatment are followed for the planned study duration (“retrieved dropout”): their post-discontinuation events and exposure contribute directly to the NB likelihood. When within-subject rates may vary over calendar time (e.g., due to seasonality), the `nb_sim_seasonal()` simulator captures this variation in the data-generation model while the likelihood analysis retains the same observed-exposure structure.

Missing not at random (MNAR) dropout is a different setting. If the post-dropout event process depends on unobserved future outcomes or unmeasured deterioration, the observed-exposure likelihood no longer identifies the same full-follow-up estimand without additional assumptions. A natural future extension would add MNAR sensitivity analysis by imputing post-dropout events under reference-based assumptions (e.g., experimental-arm dropouts revert to control-arm rates) compatible with the Gamma–Poisson model; this is not currently implemented (National Research Council 2010; Keene, Roger, Hartley, and Kenward 2014).

3. Sample size methodology

3.1. Negative binomial model

For subject i in treatment arm g with exposure time t_i , the event count Y_i follows a negative binomial distribution with mean $\mu_i = \lambda_g t_i$ and variance $\text{Var}(Y_i) = \mu_i + k\mu_i^2$, where λ_g is the arm-specific event rate and k is the dispersion parameter. This arises from a Gamma–Poisson mixture: each subject has a latent rate $\Lambda_i \sim \text{Gamma}(\text{shape} = 1/k, \text{scale} = k\lambda_g)$, so $E[\Lambda_i] = \lambda_g$ and $\text{Var}(\Lambda_i) = k\lambda_g^2$, and, conditional on Λ_i , the event count is $\text{Poisson}(\Lambda_i t_i)$. In `MASS::glm.nb()` terminology the shape parameter $\theta_{\text{NB}} = 1/k$; thus $k = 1/\theta_{\text{NB}}$. (Throughout this paper, k always denotes the NB dispersion parameter except where explicitly noted as the number of analyses K in the **gsDesign** interface.)

3.2. Wald and score-test sample size formulas

The treatment effect is the log rate ratio $\theta = \log(\lambda_2/\lambda_1)$, estimated via maximum likelihood under a negative binomial GLM with log link and exposure offset. The design target is statistical information, $\mathcal{J} = 1/\text{Var}(\hat{\theta})$, not the raw event count alone. The Fisher information per subject is $\mathcal{J}_i = \mu_i/(1 + k\mu_i)$. If $W_g = \sum_{i \in g} \mu_i/(1 + k_g\mu_i)$, then

$$\mathcal{J} = \frac{1}{1/W_1 + 1/W_2}.$$

When exposures are summarized by arm-specific averages and the randomization ratio is $r = n_2/n_1$, this leads to the fixed-design sample size formula

$$n_1 = \frac{(z_\alpha + z_\beta)^2}{(\theta - \theta_0)^2} \left[\left(\frac{1}{\mu_1} + k_1 \right) + \frac{1}{r} \left(\frac{1}{\mu_2} + k_2 \right) \right]$$

where $\theta_0 = \log(\text{rr0})$ is the null log rate ratio ($\theta_0 = 0$ for superiority), $\mu_g = \lambda_g \bar{t}_g$, $\bar{t}_g$ is the average exposure, and k_g may be inflated by a variance correction factor $Q_g = E[t_g^2]/(E[t_g])^2$ to account for variable follow-up (Zhu and Lakkis 2014). Throughout, subscript $g = 1$ denotes the control arm and $g = 2$ the experimental arm; k_g is the arm-specific dispersion, which reduces to a common k when specified as a scalar. Accordingly, two analyses with similar event totals can carry different information if follow-up or dispersion differs.

This is the default `test_type = "wald"` calculation in `sample_size_nbinom()`. It is equivalent across the three primary references: Method 3 of Zhu and Lakkis (2014) (fixed sample size), Friede and Schmidli (2010) (blinded SSR), and Mütze *et al.* (2019) (group sequential designs). It is therefore the directly matched planning formula when the final analysis will be the Wald log-rate-ratio test. The practical difference from more familiar first-event formulas is that when the protocol imposes an event gap, the expected count μ_g must reflect the gap-corrected effective rate rather than a naive event-count proxy.

For a score test evaluated under the null, using only the alternative variance for the Type I error component is less natural. `sample_size_nbinom(test_type = "score")` therefore implements a two-variance calculation in the spirit of restricted-null score-test sample sizing (Farrington and Manning 1990):

$$n_1 = \frac{(z_\alpha \sqrt{V_0} + z_\beta \sqrt{V_1})^2}{(\theta - \theta_0)^2},$$

where

$$V_1 = \left(\frac{1}{\mu_1} + k_1 \right) + \frac{1}{r} \left(\frac{1}{\mu_2} + k_2 \right)$$

is the variance factor under the planned alternative and V_0 is the corresponding null variance factor under the constrained null rate ratio $\exp(\theta_0) = \text{rr0}$. The alpha term is therefore calibrated on the null information scale, while the beta term remains calibrated on the planned alternative information scale.

This distinction leads to a practical recommendation. The Zhu–Lakkis / Friede–Schmidli / Mutze formula should remain the baseline for a Wald primary analysis and for moderate-information designs where simulation confirms Type I error. A score statistic for the analysis and operating-characteristic simulations is often the more appropriate choice when any of the following conditions hold:

- total sample size is small (e.g., fewer than ~ 200 per arm),
- dispersion is high ($k \geq 1$) or event gaps are large (≥ 20 days),
- the null margin differs from 1 (non-inferiority or super-superiority), or
- adaptive or group sequential decisions make Type I calibration central.

The two-variance score formula is useful as a test-aligned alternative and diagnostic, but it is not automatically the most powerful choice. For superiority designs like those studied below, the traditional Wald sample size paired with the score test can be a defensible practical default because it preserves Type I error and provides a small sample-size margin. Final designs should therefore compare Wald and score sizing under the planned score test rather than assuming one formula is uniformly preferable. In practice, this comparison is best made through simulation, as illustrated in the group sequential example below.

In `sample_size_nbinom()`, the same information framework is used for superiority, non-inferiority, and super-superiority designs via the `rr0` argument, and the nuisance parameters may be specified either as common scalars or as arm-specific vectors.

3.3. Jensen’s inequality correction for event gaps

In trials with a mandatory gap of duration γ after each counted event — representing the total exclusion window that may include event duration (e.g., a course of treatment) plus a protocol-defined separation period — the effective counted-event rate for a subject with underlying Poisson rate λ is $\lambda_{\text{eff}} = \lambda/(1 + \lambda\gamma)$ (Cox and Lewis 1966). This follows from elementary renewal theory: expected events per unit calendar time equal the reciprocal of the expected inter-event interval $E[T] = 1/\lambda + \gamma$. However, applying this expression at the population rate is a population-level average that ignores subject-level heterogeneity. Since $f(x) = x/(1 + x\gamma)$ is concave and subject rates Λ_i are random (Gamma-distributed), Jensen’s inequality gives $E[f(\Lambda)] < f(E[\Lambda])$.

A second-order delta-method expansion around λ ,

$$E[f(\Lambda)] \approx f(\lambda) + \frac{1}{2}f''(\lambda) \text{Var}(\Lambda),$$

with $f''(\lambda) = -2\gamma/(1 + \lambda\gamma)^3$ and $\text{Var}(\Lambda) = k\lambda^2$, yields

$$\lambda_{\text{eff}} \approx \frac{\lambda}{1 + \lambda\gamma} \left(1 - \frac{k\lambda\gamma}{(1 + \lambda\gamma)^2} \right).$$

Table 4 shows the correction magnitude. When the first-order quantity $k\lambda\gamma/(1 + \lambda\gamma)^2$ approaches 0.25, as in the first row, the second-order expansion is near its limit of accuracy and the full Gamma–Poisson expectation would differ modestly from the displayed value; the correction still moves λ_{eff} in the correct direction and reduces simulation-based approximation error (Table 5).

λ	k	γ	Correction (%)
1.0	1.0	1.0	25.0
1.0	1.0	0.5	22.2
0.5	1.0	1.0	22.2
0.3	1.0	1.0	17.8
0.5	1.0	0.5	16.0
1.0	0.5	1.0	12.5
0.3	1.0	0.5	11.3
1.0	0.5	0.5	11.1

Table 4: Jensen correction magnitude for selected parameter combinations.

A simulation study with 10,000 replicates per scenario confirmed that the corrected formula reduces the effective rate estimation error from approximately 8% to 1% and produces slightly conservative power (Table 5). During package development, this correction was needed for simulation-based power to agree with the analytical planning formula when event gaps were non-negligible.

In the implementation, the same gap rule is used by `nb_sim()` and `cut_data_by_date()`: after each counted event, the next event cannot occur until at least γ time units later (this is enforced during event generation, not applied as a post-hoc filter), and the corresponding gap time is removed from the at-risk set used for information calculations. The rate λ is therefore interpreted as the rate per unit of *at-risk* time (excluding recovery), not per unit of calendar time. An alternative approach — simply reducing the exposure offset by the gap periods without modelling the gap in the event-generation process — would understate the effective rate and produce a different (generally less accurate) power calculation.

k	Gap (days)	n (corrected)	n (naive)	Power (corr.)	Power (naive)
0.5	20	436	422	0.9061	0.8992
1.0	20	712	684	0.9170	0.9033
0.5	30	454	436	0.9042	0.8934
1.0	30	740	700	0.9105	0.8988

Table 5: Power comparison: Jensen-corrected vs naive formula (10,000 replicates each).

Broader parameter sweep

The four scenarios in Table 5 fix $\lambda_1 = 0.4$ and vary only k and the gap duration. To characterize *when* the correction matters across a wider range of clinically relevant settings, we conducted a broader simulation sweep over three axes: the control event rate λ_1 , the dispersion k , and the event gap γ . All scenarios target a single rate ratio $\lambda_2/\lambda_1 = 0.7$ (30% reduction); the sweep therefore isolates the impact of the correction on power at a fixed effect size.

Parameter grid. The sweep covers $4 \times 4 \times 4 = 64$ scenarios defined by:

Parameter	Values	Clinical rationale
Control rate	0.3, 0.5, 1.0, 2.0 events/month	Low to high recurrent-event burden
Dispersion	0.1, 0.3, 0.5, 1.0	Near-Poisson to substantial patient heterogeneity
Event gap	10, 20, 30, 45 days	Short to extended counting windows

Table 6: Broad sweep parameter grid for Jensen correction impact study. The sweep extends to $k = 1.0$; in disease areas with higher dispersion ($k \geq 2$, e.g., severe COPD exacerbations or asthma), the correction is even larger and simulation becomes the primary validation tool because the second-order delta-method expansion loses accuracy.

The remaining design parameters are held constant across all scenarios: $\lambda_2/\lambda_1 = 0.7$ (30% rate reduction), power = 90%, one-sided $\alpha = 0.025$, dropout rate = $0.1/12/\text{month}$, maximum follow-up = 12 months, trial duration = 24 months, and piecewise accrual (rate R for 0–6 months, $2R$ for 6–12 months). These are fixed because they affect power symmetrically for both the corrected and naive designs, so they shift both power estimates together without meaningfully changing the *gap* between them. The three chosen axes (λ_1, k, γ) correspond directly to the three terms in the correction factor:

$$1 - \frac{k\lambda_1\gamma}{(1 + \lambda_1\gamma)^2}$$

Rationale for disease-area calibration. The parameter ranges are informed by published recurrent event trials. Heart failure hospitalization trials such as PARADIGM-HF report control rates of approximately 0.3 events/month with dispersion near 0.5 (McMurray *et al.* 2014). COPD exacerbation trials (e.g., FLAME, IMPACT) report rates around 0.5–0.8/month with $k \in [0.5, 1.0]$ and commonly define a new exacerbation as requiring ≥ 28 symptom-free days (Wedzicha *et al.* 2016; Lipson *et al.* 2018); in practice, lower dispersion ($k < 0.4$) tends to co-occur with larger gap definitions, and higher values ($k \geq 2$) are observed with severe COPD exacerbations and asthma endpoints. Multiple sclerosis relapse trials observe rates of 1–2/year with moderate-to-high overdispersion (Hauser *et al.* 2017). Some published analyses (e.g., IMPACT) effectively assume zero event duration by using start-date-to-start-date counting and an always-at-risk exposure offset; the `event_gap` parameter in `gsDesignNB` can accommodate either convention, as described in Section 2.3. At one extreme ($\lambda_1 = 0.3$, $k = 0.1$, gap = 10 days), the theoretical correction factor is below 0.1% and the naive formula suffices. At the other ($\lambda_1 = 2.0$, $k = 1.0$, gap = 45 days), it exceeds 10%, and the underpowering from the naive formula becomes practically meaningful.

Simulation protocol. For each scenario, we compute both the Jensen-corrected and naive sample sizes, simulate 3,500 trials at the corrected n , and extract results for the first n_{naive} subjects from the same simulated data. This paired design makes the power comparison

very precise ($SE \approx 0.5$ percentage points at 90% power). The aggregated results — scenario parameters, sample sizes, empirical power, paired power differences, and discordant pair counts — are saved for use in downstream analyses.

Results. Table 7 summarizes selected scenarios from the 64-scenario sweep, ordered by the theoretical correction factor. The corrected design achieves power within simulation error of the 90% target across all 64 scenarios (range: 0.896–0.924), while the naive design falls below 90% in the majority of scenarios where $k \geq 0.5$ and the gap exceeds 20 days.

λ_1	k	Gap (days)	n (corr.)	n (naive)	Δn	Power (corr.)	Power (naive)	Δ Power (pp)
1.0	1.0	30	426	406	20	0.911	0.899	1.1
0.5	1.0	45	490	456	34	0.910	0.890	2.1
1.0	1.0	45	446	420	26	0.911	0.896	1.5
2.0	1.0	30	402	388	14	0.911	0.900	1.0
0.5	1.0	30	466	442	24	0.914	0.893	2.1
0.3	1.0	45	542	502	40	0.919	0.903	1.6
2.0	1.0	45	418	402	16	0.911	0.899	1.2
0.3	1.0	30	516	488	28	0.918	0.904	1.4
0.5	0.5	45	300	284	16	0.908	0.891	1.7
0.3	0.5	45	348	332	16	0.911	0.895	1.7
2.0	0.1	45	96	96	0	0.918	0.918	0.0
0.3	0.1	10	162	162	0	0.900	0.900	0.0

Table 7: Selected scenarios from the broad sweep (3,500 replicates each). Rows are ordered from largest to smallest correction factor; the last two rows illustrate scenarios where the correction has no effect.

Three patterns emerge:

1. **The correction is driven by the $k \times \gamma$ interaction.** Dispersion $k \geq 0.5$ combined with gaps ≥ 30 days consistently produces 1–2 percentage points of underpowering from the naive formula, corresponding to 14–40 additional subjects (3–8% of total n).
2. **High event rates do not amplify the power gap as much as high dispersion.** At $k = 0.3$, even $\lambda_1 = 2.0$ with a 45-day gap shows only ~ 0.6 pp of underpowering. Conversely, at $k = 1.0$, $\lambda_1 = 0.5$ with a 30-day gap shows 2.1 pp — the largest gap in the sweep. This is because the correction factor $k\lambda\gamma/(1 + \lambda\gamma)^2$ is bounded in λ (it approaches k/γ for large λ) but grows linearly in k .
3. **The correction is negligible when either k or γ is small.** All scenarios with $k = 0.1$ show $\Delta n \leq 4$ and $\Delta \text{Power} < 0.8$ pp. Similarly, all 10-day gap scenarios show $\Delta n \leq 10$ regardless of k .

These findings provide practical guidance: the Jensen correction is most valuable for trials in therapeutic areas with substantial patient heterogeneity ($k \geq 0.5$) and protocol-defined event

gaps exceeding 20 days, such as COPD exacerbation studies with 28-day new-event windows or heart failure trials with extended hospitalization-free windows. The correction addresses only the planning formula; it does not resolve the Wald test's tendency toward Type I error inflation in finite samples. When event gaps are present, the Jensen-corrected sample size should therefore be paired with the score test for the final analysis, which controls Type I error across the scenarios studied below. Power under this combination should be confirmed by simulation.

4. Group sequential design, monitoring, and sample size re-estimation

4.1. Calendar-time design construction

The `gsNBCalendar()` function creates a group sequential design with K planned analyses from a `sample_size_nbinom_result` by evaluating the planned design at user-specified calendar analysis times. (The `gsDesign` argument `k` denotes the number of analyses K ; it should not be confused with the NB dispersion parameter k used elsewhere in this paper.) For each look it recomputes expected enrollment, exposure, events, and variance under the planned accrual and follow-up structure, then stores the corresponding statistical information in the `n.I` slot inherited from `gsDesign`. The resulting `gsNB` object therefore behaves like a `gsDesign` object for spending functions and boundary summaries while retaining the NB-specific quantities needed for simulation and interpretation.

This calendar-time emphasis matters because timing of analyses in trials is often primarily governed by operational issues, and thus we have chosen this approach to analysis timing. This also makes computation faster since there is no root-finding for when analyses are expected. Finally, calendar times are easily adjusted if the information fractions derived from this approach are thought inappropriate. If C_j denotes a calendar floor for look j , δ_j a minimum separation from the prior look, and $\mathcal{J}_j$ the target information for that look, the analysis time for $j = 1, \dots, K$ can be expressed schematically as

$$\tau_j = \max \left\{ C_j, \tau_{j-1} + \delta_j, \inf \left\{ t : \hat{\mathcal{J}}(t) \geq \mathcal{J}_j \right\} \right\}.$$

`gsNBCalendar()` addresses the planning side of this problem through `analysis_times`, while `get_cut_date()` applies the same logic to simulated or observed trial data by combining calendar, event-count, completer, and information targets. This is the package's bridge between textbook information-time group sequential design and the calendar rules encountered in real monitoring charters.

4.2. Interim data cuts and information estimates

The function `cut_data_by_date()` converts subject-level recurrent-event histories into an interim analysis data set with one row per randomized subject, containing counted events, time at risk, and total follow-up. The same interface is implemented for both `nb_sim()` and `nb_sim_seasonal()`. `get_analysis_date()` solves the special case of event-driven

looks, whereas `cut_date_for_completers()` and `cut_completers()` support analyses defined by the number of subjects who have completed their planned follow-up. More generally, `get_cut_date()` takes the maximum of whichever criteria are supplied, allowing hybrid rules such as “no earlier than month 8 and only after 60 blinded information units.”

Because information depends on exposure and dispersion, the package distinguishes raw event totals from the quantities actually used for monitoring. `calculate_blinded_info()` implements the pooled negative binomial fit of Friede and Schmidli (2010), then splits the pooled rate according to the planned rate ratio. `estimate_nb_mom()` supplies method-of-moments estimates that are useful when maximum likelihood fits become unstable in sparse or highly overdispersed interim data.

4.3. Wald and score inference with a two-sided ML/MoM fallback rule

For unblinded interim analyses, `mutze_test()` fits the negative binomial log-linear model with an offset for exposure and returns either a Wald statistic for the treatment effect (`test_type = "wald"`) or a score statistic evaluated under the null (`test_type = "score"`). The Wald statistic is the historical analysis model used by Mütze *et al.* (2019) and by the package’s earlier GS simulations. The score statistic fits only the null model, so the numerator and variance are both evaluated on the null information scale; this is the test aligned with the two-variance sample-size formula above. When the fitted NB shape parameter $\theta_{\text{NB}} = 1/\hat{k}$ is well-behaved, the ML Wald or score test is used directly. When it is not, `mutze_test()` applies one of two *statistically principled* fallbacks rather than a blind Poisson substitution:

- **Near-Poisson regime** ($\hat{k} < 1/\text{poisson_threshold}$, default $\hat{k} < 0.02$): the fitted NB is numerically indistinguishable from a Poisson and the function falls back to the corresponding Poisson Wald or score test. This handles the common case of `MASS::glm.nb()` drifting toward its iteration ceiling when the data truly are Poisson.
- **Extreme-overdispersion or ML-failure regime** ($\hat{k} > \text{mom_threshold}$, default $\hat{k} > 20$; or `glm.nb()` non-convergence): the function re-estimates $(\hat{\lambda}_1, \hat{\lambda}_2, \hat{k})$ by method of moments via `estimate_nb_mom()` and plugs those values into the same observed Fisher information formula that the package uses everywhere else,

$$\hat{\mathcal{J}} = \frac{1}{1/W_1 + 1/W_2}, \quad W_g = \sum_{i \in g} \frac{\hat{\mu}_{g,i}}{1 + \hat{k} \hat{\mu}_{g,i}}, \quad \hat{\mu}_{g,i} = \hat{\lambda}_g t_i.$$

The Wald statistic is then $\hat{\theta}/\sqrt{1/W_1 + 1/W_2}$ with $\hat{\theta} = \log(\hat{\lambda}_2/\hat{\lambda}_1)$; the score statistic uses the corresponding null score and null information.

The methodological point of the second branch is straightforward. Under genuine overdispersion ($k \gg 0$), a Poisson fallback underestimates $\text{Var}(\hat{\theta})$ by a factor of $(1 + k\mu)^{-1}$ per subject, inflating Type I error exactly when the NB model is most needed. MoM does not require ML convergence, always returns a nonnegative $\hat{k}$, and, plugged into the NB information formula, yields a variance estimate with the correct overdispersion structure. The same two-branch rule is used in `calculate_blinded_info()` (pooled blinded estimation) and `unblinded_ssr()` (treatment-specific estimation for SSR), so that every entry point in the package handles fit failure in the same statistically sensible way. Each of these functions now returns a **fallback**

field ("ml", "mom", or "poisson") so that simulation engines and diagnostic tables can record which estimator was used at each interim.

The simulation engines also expose blinded and unblinded information columns under both ML and MoM estimation (e.g., `info_blinded_ml`, `info_unblinded_mom`) so that users can stress-test whether a design's monitoring behaviour is sensitive to the nuisance-parameter estimation method. These alternative columns are diagnostics rather than competing estimands; in well-behaved data they agree closely, while in sparse or unstable interim data the MoM columns remain finite when the ML columns degenerate.

4.4. Blinded and unblinded SSR

At interim analyses, nuisance parameters (k and the underlying event rates) can be re-estimated to update the required sample size. `blinded_ssr()` uses the pooled model of [Friede and Schmidli \(2010\)](#), preserving masking but typically producing a more conservative adapted sample size. `unblinded_ssr()` instead fits treatment-specific rates and a common dispersion to the interim data, while holding the target treatment effect at its planned value for the final sample size calculation.

Both SSR routines account for dropout in the same exposure-based sense as the fixed-design calculation. The interim nuisance estimates use the observed `tte` supplied by `cut_data_by_date()`, so subjects who drop out before the interim contribute their observed exposure and events. The recalculated total sample size is then obtained by calling `sample_size_nbinom()` with the planned accrual, dropout, maximum follow-up, and event-gap assumptions. The resulting SSR therefore adjusts for the loss of information expected from dropout under the specified ignorable censoring model; it is not an MNAR sensitivity procedure.

To prevent extreme sample-size inflation in unfavourable scenarios, `sim_ssr_nbinom()` caps the adapted enrollment at `n_max`, which defaults to 150% of the planned total and is always at least the planned sample size. This cap is applied after the GS inflation factor so that it bounds the operational commitment while still allowing meaningful upward adaptation.

`sim_ssr_nbinom()` wraps these choices into a full adaptive simulation engine. It supports "No adaptation", "Blinded SSR", and "Unblinded SSR" strategies; searches for interim looks using information-based timing; and allows efficacy and futility bounds to be driven by alternative information measures through `bound_info = "unblinded_ml", "blinded_ml", "unblinded_mom", or "blinded_mom"`. In the simulation study below, bounds are computed with spending functions using `usTime = lsTime = min(planned IF, actual IF)` at the interim so that spending is not accelerated when more information than expected is observed. Futility assessment is deferred until at least 30% of the planned information is available.

From a computing perspective, both `sim_gs_nbinom()` and `sim_ssr_nbinom()` are designed for large Monte Carlo studies: they use `data.table` for repeated aggregation, support reproducible parallel execution, and return both trial-level and analysis-level summaries so that design calibration can be checked after the fact.

5. Reproducible workflows

5.1. Example 1: fixed design to calendar-time GS simulation

The first workflow shows the main path from fixed-design planning to a calendar-time GS simulation. The full executable version appears in the group-sequential simulation vignette; the code here is a minimal example.

```
library(gsDesignNB)

enroll_rate <- data.frame(rate = 12, duration = 6) # enrollment: 12 subjects/month for 6
fail_rate <- data.frame(                          # event rates (fail_rate follows simt
  treatment = c("Control", "Experimental"),
  rate = c(0.6, 0.42),                          # NB event rate (events/month)
  dispersion = c(0.4, 0.4)                        # NB dispersion parameter k
)
dropout_rate <- data.frame(
  treatment = c("Control", "Experimental"),
  rate = c(0.05, 0.05),                          # monthly dropout rate
  duration = c(12, 12)
)

fixed_design <- sample_size_nbinom(
  lambda1 = 0.6,
  lambda2 = 0.42,
  dispersion = 0.4,
  power = 0.8,
  alpha = 0.025,
  accrual_rate = enroll_rate$rate,
  accrual_duration = enroll_rate$duration,
  trial_duration = 12,
  max_followup = 6
)

gs_design <- gsNBCalendar(
  fixed_design,
  k = 3,                                           # K = 3 analyses
  test.type = 4,
  analysis_times = c(4, 8, 12)
)

# n_sims = 50 is for illustration only; the GS verification vignette uses
# 3,600 replicates for its cached operating-characteristic results.
sim_gs <- sim_gs_nbinom(
  n_sims = 50,
  enroll_rate = enroll_rate,
```

```

    fail_rate = fail_rate,
    dropout_rate = dropout_rate,
    max_followup = 6,
    n_target = ceiling(utils::tail(gs_design$n_total, 1)),
    design = gs_design,
    cuts = lapply(gs_design$T, function(t) list(planned_calendar = t)),
    seed = 20260413
  )

oc <- summarize_gs_sim(
  check_gs_bound(sim_gs, gs_design, info_col = "info_unblinded_ml")
)

oc$analysis_summary

```

The `analysis_summary` element of `oc` contains per-look cumulative rejection and futility probabilities, mean observed information, and mean calendar time at each look, so that the simulated operating characteristics can be compared directly to the planned boundaries. This example illustrates the main software-design principle of the package: the fixed design determines the target information, `gsNBCalendar()` maps that information to calendar looks, `sim_gs_nbinom()` generates trial histories consistent with the planning assumptions, and `check_gs_bound()` applies the group sequential boundaries on the selected information scale.

5.2. Example 2: information-based SSR

The second workflow shows how the same design object can be used in an adaptive simulation with blinded and unblinded sample size re-estimation.

```

library(gsDesignNB)

ssr_res <- sim_ssr_nbinom(
  n_sims = 50,
  enroll_rate = enroll_rate,
  fail_rate = fail_rate,
  dropout_rate = dropout_rate,
  max_followup = 6,
  design = gs_design,
  strategies = c("No adaptation", "Blinded SSR", "Unblinded SSR"),
  bound_info = "unblinded_ml",
  seed = 20260413
)

summarize_ssr_sim(ssr_res)$trial_summary[
  , c("strategy", "rejection_rate", "mean_n", "pct_adapted")
]

```

The SSR simulation-study article uses this same engine to define a 90-scenario grid and adds design constraints on when adaptation is allowed. The bundled package cache keeps the

non-null power grid deliberately small for portable documentation builds, while the null Type I calibration tables are stored at production scale. The essential software idea is already visible in the compact example: one `gsNB` planning object can drive both non-adaptive and adaptive operating-characteristic evaluations.

5.3. Additional workflows

The package includes further worked examples that round out the software contribution.

Workflow	Main functions	What it adds
Seasonal recurrent-event simulation	<code>nb_sim_seasonal()</code> , <code>cut_data_by_date()</code>	Calendar-time variation in failure rates
Completer-based monitoring	<code>cut_date_for_completers()</code> , <code>cut_completers()</code>	Interim looks keyed to completed follow-up
Non-inferiority and super-superiority	<code>sample_size_nbinom(rr0 = ...)</code>	Alternative null hypotheses in the same design framework
Blinded-information diagnostics	<code>calculate_blinded_info()</code> , <code>estimate_nb_mom()</code>	Stability checks for sparse or unstable interims
Verification studies	<code>nb_sim()</code> , <code>mutze_test()</code> , <code>sim_gs_nbinom()</code>	Simulation-based agreement checks for design formulas

These workflows are documented in the package vignettes so that the manuscript examples can remain brief while the installed package still serves as the full replication resource.

6. Simulation study

6.1. Design

The manuscript reports selected operating characteristics, while the package articles serve as executable supplementary material. This keeps the paper readable in a software-journal format but still makes the simulation design, code, and fuller numerical output available to readers. In particular, the `verification-simulation`, `simulation-example`, `score-vs-wald-simulation`, and `ssr-simulation-study` articles contain the code used to reproduce the verification checks, event-gap studies, score-versus-Wald calibration, and SSR operating characteristics summarized below.

The `ssr-simulation-study` article defines a 90-scenario grid varying the control rate $\lambda_1 \in \{0.3, 0.5, 0.8\}$ /month, dispersion $k \in \{0.5, 1.0\}$, planned accrual rate $\in \{12, 18, 24\}$ /month, and true rate ratio $RR \in \{0.6, 0.7, 0.85, 1.0, 1.1\}$ (ninety combinations in total). The planned design was built with `sample_size_nbinom()` and `gsNBCalendar()`, targeting 90% power for $RR = 0.7$ with $k = 0.5$, $\lambda_1 = 0.5$ /month, and planned accrual of 26/month, for a planned total sample size of 312 and a maximum follow-up of 12 months. The bundled article cache uses 3,600 replicates per $RR < 1$ power scenario, 20,000 replicates per $RR = 1$ null scenario, and 1,000

replicates per $RR > 1$ interior-null scenario. The dedicated non-binding Type I tables use an additional 20,000 replicates per dispersion and test statistic. See the `ssr-simulation-study` vignette for the full scenario grid, replicate controls, and cache-regeneration code.

Three strategies were compared: (1) no adaptation, (2) blinded SSR, and (3) unblinded SSR, all using a group sequential design with Hwang–Shih–DeCani spending functions (Hwang, Shih, and de Cani 1990).

Table 9 summarizes how the Monte Carlo evidence is divided between the manuscript and the package supplement. For a rejection probability $\hat{p}$ based on B replicates, the Monte Carlo standard error is $\sqrt{\hat{p}(1 - \hat{p})/B}$; this is reported or bounded for the main simulation summaries.

Component	Manuscript role	Supplementary package article	Replicates and reported quantities
Fixed-design verification	Demonstrate agreement between NB planning formulas and simulated power	<code>verification-simulation</code> , <code>sample-size-nbinom</code>	Scenario-specific power, information, and Monte Carlo error checks
Event-gap correction	Show where the Jensen correction changes sample size and power	<code>simulation-example</code> , <code>verification-simulation</code>	64-scenario sweep; 3,500 or 10,000 replicates depending on table; corrected vs naive n and power
Wald versus score inference	Compare Wald and score tests under Wald- and score-sized designs	<code>score-vs-wald-simulation</code>	27-scenario factorial grid; 20,000 null replicates and 3,500 power replicates per cell; Type I error, power, and null Z distributions
SSR Type I calibration	Summarize null rejection and adapted sample size	<code>ssr-simulation-study</code>	20,000 replicates for $RR = 1$; rejection rate, MCSE, mean n , and percent adapted
SSR power recovery	Compare no adaptation, blinded SSR, and unblinded SSR under nuisance misspecification	<code>ssr-simulation-study</code>	Bundled production cache uses 3,600 replicates per $RR < 1$ scenario and 1,000 replicates per $RR > 1$ scenario

Component	Manuscript role	Supplementary package article	Replicates and reported quantities
SSR starting-size sensitivity	Compare Wald- and score-sized starting designs under the score final test	<code>ssr-simulation-study</code>	Low-event stress setting; 5,000 null and 3,000 planned-effect replicates per starting-size rule
Estimation diagnostics	Check whether ML, MoM, and fallback labels affect monitoring conclusions	<code>blinded-info-diagnostics</code> , <code>ssr-simulation-study</code>	Blinded/unblinded information paths, nuisance estimates, and estimator fallback labels

Table 9: Simulation reporting map. The package articles are treated as executable supplementary material for the condensed results reported in the manuscript.

6.2. Results

Wald versus score calibration. The `score-vs-wald-simulation` article evaluates 27 base scenarios varying $\lambda_1 \in \{0.15, 0.40, 1.00\}$, $k \in \{0.2, 0.5, 1.0\}$, and event gaps of 0, 15, or 30 days, with both Wald- and score-sized designs. In this grid, score sizing changed the total sample size by at most four subjects and was never larger than Wald sizing, because the null variance was smaller than the alternative variance for these superiority assumptions. The empirical null results therefore isolate the effect of the test statistic more than the effect of sample size: Wald-test Type I error ranged from 0.0243 to 0.0317, with 13–15 of 27 scenarios above nominal beyond Monte Carlo error depending on the sizing method; score-test Type I error ranged from 0.0200 to 0.0264, with no scenario above nominal beyond Monte Carlo error and several conservative cells. Under $RR = 0.70$, Wald-test power averaged about 0.904–0.907. Score-test power averaged 0.895 under Wald sizing and 0.893 under score sizing; 8–9 of 27 score-test scenarios were below the 90% target beyond Monte Carlo error. These results support using the score test when Type I error preservation is the priority, while treating Wald sizing as a useful practical baseline or power margin for score-test superiority designs. Score-test designs should still be simulated for power and may need a modest information margin.

SSR Type I calibration. Table 10 summarizes the empirical one-sided Type I error under $RR = 1.0$ for the two null dispersion configurations at nominal $\alpha = 0.025$, comparing the Wald statistic with the score statistic under the same group sequential design. With 20,000 replicates per cell the Monte Carlo standard error is about 0.001. The Wald statistic exhibits mild Type I error inflation, with empirical rejection rates ranging from 0.0279 to 0.0312. The score statistic is conservative in the same design, with empirical Type I error ranging from 0.0214 to 0.0232. Thus the preferred remedy is to align the analysis and simulation engine with the score test and then verify power, rather than to lower alpha for a Wald analysis.

Test	k	Strategy	Type I	Mean n	% adapted
Wald	0.5	No adaptation	0.0306	312.0	0.0
Wald	0.5	Blinded SSR	0.0309	320.5	33.2
Wald	0.5	Unblinded SSR	0.0309	325.4	45.3
Wald	1.0	No adaptation	0.0279	312.0	0.0
Wald	1.0	Blinded SSR	0.0307	508.4	98.4
Wald	1.0	Unblinded SSR	0.0312	519.5	98.4
Score	0.5	No adaptation	0.0229	312.0	0.0
Score	0.5	Blinded SSR	0.0229	320.6	33.4
Score	0.5	Unblinded SSR	0.0232	325.4	45.5
Score	1.0	No adaptation	0.0214	312.0	0.0
Score	1.0	Blinded SSR	0.0229	509.5	99.0
Score	1.0	Unblinded SSR	0.0232	520.5	99.0

Table 10: Empirical one-sided Type I error and average adapted sample size under the null ($RR = 1.0$), 20,000 replicates per cell. Futility stopping is ignored in this non-binding calibration table so all trials continue unless they cross an efficacy boundary.

Power recovery at the designed effect ($RR = 0.7$). In the production score-test cache, no-adaptation power ranged from 0.698 to 0.896 across nuisance scenarios and averaged 0.807. Blinded and unblinded SSR both ranged from about 0.880 to 0.903 and averaged 0.893. The recovery was most important under the higher-dispersion scenarios: for $k = 1.0$, mean no-adaptation power was 0.733, whereas blinded and unblinded SSR averaged 0.889. Because the score test is conservative, designs using it may still need a modest information margin to achieve a 90% power target across the full nuisance range.

Diminishing returns at smaller effects. For $RR = 0.85$ (true effect smaller than planned), the planned design is inherently underpowered; SSR compensates for nuisance-parameter misspecification but cannot overcome the effect-size mismatch. In the production score-test cache, no-adaptation power averaged 0.242, while blinded and unblinded SSR averaged about 0.296 and 0.297. However, futility stopping provides substantial cost savings: approximately 46% of trials stopped for futility (38% at the first interim, 8% at the second), reducing the average sample size from the planned 312 to about 291 without adaptation or about 345 with SSR. Under the null ($RR = 1.0$), futility stopping was even more effective, with 87% of trials stopped early and average sample size reduced to about 268–285 depending on strategy. For $RR = 1.1$ (treatment worse than control), all three strategies had maximum rejection rates of 0.005 across the scenario grid, confirming that SSR does not artificially inflate one-sided rejections in the wrong direction.

Blinded versus unblinded SSR. In this score-test configuration, blinded and unblinded SSR have very similar operating characteristics. At $RR = 0.7$ their average power differs by less than 0.001 and their average sample sizes are both about 351 subjects. Under the non-binding null calibration, both strategies adapt frequently when $k = 1.0$, and unblinded SSR has slightly larger mean sample size in the displayed scenarios. The practical conclusion is therefore not that one approach is uniformly more efficient, but that the masked and unmasked

update rules should be compared under the trial-specific nuisance range, adaptation cap, and analysis statistic.

A potential concern with blinded SSR is unnecessary sample-size inflation when the true treatment effect is larger than planned, because the pooled nuisance estimates may overstate the required information. However, early efficacy stopping largely neutralizes this risk. At $RR = 0.6$ (a larger effect than the planned $RR = 0.7$), 94% of trials crossed an efficacy boundary at an interim analysis, SSR was triggered in fewer than 5% of trials, and the median sample size was identical to the no-adaptation design; mean sample size increased by only about 11 subjects (312 vs. 323). Even at the planned $RR = 0.7$, early efficacy stopping at 67% of trials limits the average inflation from SSR to about 43 subjects. At higher dispersion ($k = 1.0$), where blinded SSR is triggered more often, the pattern holds: at $RR = 0.6$, SSR was applied in about 9% of trials, but early efficacy stopping still occurred in 91%, keeping the median sample size at 312 and the mean at 334 compared with 312 without adaptation. The additional enrollment mostly occurred in the 5% of trials that reached the 95th percentile ($n = 564$), which are precisely the trials where nuisance misspecification would otherwise have caused the design to lose power. At the planned $RR = 0.7$ with $k = 1.0$, SSR was applied in 35% of trials and mean sample size rose to 393; however, power increased from 0.733 without adaptation to 0.889, confirming that the inflation is productive rather than wasteful.

SSR starting-size sensitivity. The fixed-design score-versus-Wald sweep suggests that the traditional Wald sample size can provide a small information margin when the planned analysis uses the score test. To check whether that margin carries into SSR, the SSR article adds a targeted low-event stress setting with $\lambda_1 = 0.15$, $k = 0.5$, $RR = 0.7$, no event gap, and the same score final test. The Wald-sized group sequential design has fixed-design $n = 390$ and final group sequential $n = 472$, compared with 384 and 464 for the score-sized design. With 5,000 non-binding null replicates per starting-size rule, score-test Type I error remains close to nominal for both starts: 0.0252–0.0258 for the Wald-sized start and 0.0258–0.0266 for the score-sized start (MCSE about 0.0022). With 3,000 planned-effect replicates per starting-size rule, power is also similar: 0.898–0.907 for the Wald-sized start and 0.900–0.911 for the score-sized start (MCSE about 0.005). Thus Wald sizing should be viewed as a candidate margin to evaluate, not as a guaranteed SSR power improvement.

7. Discussion

The **gsDesignNB** package fills a gap in the R ecosystem for planning and adapting recurrent-event trials with NB outcomes. Its statistical contributions are the alignment of fixed-design planning, calendar-time monitoring, and sample size re-estimation around Fisher information for the log rate ratio; the Jensen correction for event gaps with accompanying simulation evidence; and a two-sided ML/MoM fallback rule that keeps Wald inference well-defined under both near-Poisson and extreme-overdispersion regimes. Its computing contributions are the object flow from `sample_size_nbinom_result` to `gsNB` to simulation outputs; subject-level recurrent-event simulation with event-gap-aware interim cuts; diagnostic information calculations under ML and method-of-moments nuisance estimation; and reproducible Monte Carlo engines for both fixed and adaptive monitoring strategies.

The Jensen correction is important precisely because the software is designed to keep planning

and simulation on the same scale. When event gaps are non-negligible, the naive renewal approximation overstates the effective rate and therefore overstates information. The 64-scenario sweep confirms that the correction is most valuable when dispersion and event-gap duration are both substantial: at $k \geq 0.5$ with gaps ≥ 30 days, the naive formula underpowers by about 1–2 percentage points, whereas at $k = 0.1$ or gaps ≤ 10 days the correction adds little to the required sample size. Because the same gap rule is carried into simulation and interim cutting, users can see directly when the correction matters for operating characteristics rather than only for algebra.

The score-versus-Wald sweep clarifies how the package should be used for sample-size planning. The Zhu–Lakkis / Friede–Schmidli / Mutze calculation remains the appropriate first-line calculation for a Wald primary analysis, because its variance factor is exactly the Wald alternative variance used by that analysis. It should not, however, be presented as a universal guarantee of finite-sample Type I error control. When the design is lower-information, highly overdispersed, affected by event gaps, based on a non-unity null margin, or adaptively monitored, a score-test primary analysis is often the more defensible default. The corresponding best practice is to simulate the score statistic directly and compare candidate sample-size rules under that statistic. The two-variance score formula uses the null variance V_0 for the alpha component and the planned alternative variance V_1 for the beta component, but in the superiority grid it was slightly smaller than Wald sizing and slightly less powerful for the score test. Thus Wald/Zhu–Lakkis sizing paired with the score test is a reasonable practical recommendation when it supplies a modest information margin. Because the score test can be conservative, the final design should still be calibrated for both Type I error and power; increasing the target power or total information slightly may be preferable to reverting to a Wald test with inflated Type I error.

The SSR study likewise shows how the package’s computational layer supports statistical design checks. The production score-test grid confirms that adverse nuisance-parameter misspecification can be diagnosed and that SSR moves sample size in the expected direction, but it also shows why the final test statistic matters. The non-binding null simulations show that Wald analyses exhibit mild Type I error inflation in this configuration, whereas score analyses are conservative at the same nominal one-sided alpha. The updated SSR article therefore compares Wald and score analyses directly rather than treating an ad hoc alpha reduction as the primary remedy. The larger adapted sample sizes under high dispersion should be interpreted primarily as nuisance-parameter misspecification and adaptation-cap behavior, not as a benefit of the Wald test. The score test is important because it controls rejection after SSR has increased information; it does not remove the need to simulate how often the adaptation rule increases sample size. The targeted starting-size sensitivity reinforces that distinction: an eight-subject Wald starting-size margin did not yield a clear SSR power gain, so starting-size margins should be evaluated inside the planned adaptation rule. Blinded and unblinded nuisance updates had similar power and sample-size behavior in the score-test configuration, so their relative efficiency should be calibrated for the trial-specific adaptation rule. The ability to compare `bound_info` choices, ML versus method-of-moments information paths, and the new ML/MoM fallback behaviour inside the same simulation engine is a practical advantage of the package design: because each interim reports the fallback label actually used, simulation summaries can verify that adaptive decisions were driven by well-posed estimators rather than degenerate ML fits.

The package handles dropout and administrative censoring as exposure truncation under ignorable censoring (MAR), which is the setting in which observed-data NB likelihood inference is appropriate without multiple imputation. This includes the treatment-policy estimand where subjects are followed post-treatment-discontinuation (retrieved dropout) and their post-discontinuation events and exposure enter the likelihood directly. MNAR sensitivity analysis — for example, imputing that experimental-arm dropouts revert to control-arm rates for the remainder of the planned follow-up — is a natural future extension of the simulator and data-cutting layer but is not currently implemented.

From a software perspective, **gsDesignNB** is complementary to both **gsDesign** and **gscounts**. The general-purpose **gsDesign** package provides mature spending-function and boundary calculations but does not itself supply NB recurrent-event planning formulas, calendar-time translation of those formulas, or recurrent-event simulation with blinded information and SSR. The **gscounts** implementation motivated by Mütze *et al.* (2019) addresses group sequential NB testing, whereas **gsDesignNB** emphasizes the broader workflow needed in practice: piecewise accrual and dropout, maximum follow-up, protocol event gaps with Jensen correction, seasonal simulation, completer- or information-driven interim timing, blinded as well as unblinded SSR simulation, and a two-sided ML/MoM fallback rule for unstable NB ML fits.

The package’s secondary workflows matter for a software paper even when they are not the main empirical case study. Seasonal simulation extends the constant-rate gamma-frailty model to a common time-varying setting; completer-based cuts capture another operational monitoring rule; and the non-inferiority, blinded-diagnostics, and verification vignettes show that the same core objects support multiple design questions. In that sense, the package is not only a collection of isolated functions but a coherent interface for recurrent-event trial planning and evaluation.

8. Reproducibility and software availability

The **gsDesignNB** package is available on GitHub at <https://github.com/keaven/gsDesignNB> with documentation at <https://keaven.github.io/gsDesignNB/>. The package requires R $\geq 4.1.0$ and depends primarily on R (R Core Team 2025), **gsDesignNB** (Anderson, Zhang, and Maes 2026), **gsDesign** (Anderson 2026), **data.table** (Barrett, Dowle, Srinivasan, Gorecki, Chirico, Hocking, Schwendinger, and Krylov 2026), **simtrial** (Anderson, Zhao, Blischak, and Zhang 2025), and **MASS** (Venables and Ripley 2002). Figures and rendered package documentation use **ggplot2** (Wickham 2016), **scales** (Wickham, Pedersen, and Seidel 2025b), **knitr** (Xie 2025), **rmarkdown** (Allaire, Xie, Dervieux, McPherson, Luraschi, Ushey, Atkins, Wickham, Cheng, Chang, and Iannone 2025), and **pkgdown** (Wickham, Hesselberth, Salmon, Roy, and Brueggemann 2025a).

The source distribution includes vignettes for fixed-design NB sample size calculation, calendar-time GS simulation, end-to-end SSR, the larger SSR simulation study, blinded-information diagnostics, seasonal simulation, completer-based interims, non-inferiority design, and verification of the planning formulas against simulation. We treat these articles as supplementary material for the paper: the manuscript gives the design rationale, selected tables, and interpretation, while the package articles provide the fuller grids, code paths, and rendered results. The two code examples in this paper are deliberately small extracts of those vignettes so

that the manuscript highlights the core software interface while the installed package retains more extensive executable workflows. For local exploratory use, the package also provides `run_ssr_shiny()` as an interactive front end to the SSR engine and `preview_pkgdown_site()` for documentation preview.

References

- Allaire J, Xie Y, Dervieux C, McPherson J, Luraschi J, Ushey K, Atkins A, Wickham H, Cheng J, Chang W, Iannone R (2025). *rmarkdown: Dynamic Documents for R*. R package version 2.30, URL <https://github.com/rstudio/rmarkdown>.
- Anderson K (2026). *gsDesign: Group Sequential Design*. R package version 3.9.0.9003, URL <https://keaven.github.io/gsDesign/>.
- Anderson K, Zhang H, Maes A (2026). *gsDesignNB: Sample Size and Simulation for Negative Binomial Outcomes*. R package version 0.3.2, URL <https://keaven.github.io/gsDesignNB/>.
- Anderson K, Zhao Y, Blischak J, Zhang Y (2025). *simtrial: Clinical Trial Simulation*. doi:10.32614/CRAN.package.simtrial. R package version 1.0.2, URL <https://CRAN.R-project.org/package=simtrial>.
- Anderson KM, Clark JB (2010). “Fitting spending functions.” *Statistics in Medicine*, **29**(3), 321–327. doi:10.1002/sim.3737.
- Barrett T, Dowle M, Srinivasan A, Gorecki J, Chirico M, Hocking T, Schwendinger B, Krylov I (2026). *data.table: Extension of data.frame*. doi:10.32614/CRAN.package.data.table. R package version 1.18.2.1, URL <https://CRAN.R-project.org/package=data.table>.
- Cox DR, Lewis PAW (1966). *The Statistical Analysis of Series of Events*. Methuen, London.
- Farrington CP, Manning G (1990). “Test statistics and sample size formulae for comparative binomial trials with null hypothesis of non-zero risk difference or non-unity relative risk.” *Statistics in Medicine*, **9**(12), 1447–1454. doi:10.1002/sim.4780091208.
- Friede T, Schmidli H (2010). “Blinded sample size reestimation with negative binomial counts in superiority and non-inferiority trials.” *Methods of Information in Medicine*, **49**(06), 618–624. doi:10.3414/ME09-02-0060.
- Hauser SL, Bar-Or A, Comi G, Giovannoni G, Hartung HP, Hemmer B, Lublin F, Montalban X, Rammohan KW, Selmaj K, et al. (2017). “Ocrelizumab versus interferon beta-1a in relapsing multiple sclerosis.” *New England Journal of Medicine*, **376**(3), 221–234. doi:10.1056/NEJMoa1601277.
- Hwang IK, Shih WJ, de Cani JS (1990). “Group sequential designs using a family of type I error probability spending functions.” *Statistics in Medicine*, **9**(12), 1439–1445. doi:10.1002/sim.4780091207.
- Jennison C, Turnbull BW (1999). *Group Sequential Methods with Applications to Clinical Trials*. CRC Press.

- Keene ON, Roger JH, Hartley BF, Kenward MG (2014). “Missing data sensitivity analysis for recurrent event data using controlled imputation.” *Pharmaceutical Statistics*, **13**(4), 258–264. doi:[10.1002/pst.1624](https://doi.org/10.1002/pst.1624).
- Lipson DA, Barnhart F, Brealey N, Brooks J, Criner GJ, Day NC, Dransfield MT, Halpin DMG, Han MK, Jones CE, *et al.* (2018). “Once-daily single-inhaler triple versus dual therapy in patients with COPD.” *New England Journal of Medicine*, **378**(18), 1671–1680. doi:[10.1056/NEJMoa1713901](https://doi.org/10.1056/NEJMoa1713901).
- McMurray JJV, Packer M, Desai AS, Gong J, Lefkowitz MP, Rizkala AR, Rouleau JL, Shi VC, Solomon SD, Swedberg K, Zile MR (2014). “Angiotensin–neprilysin inhibition versus enalapril in heart failure.” *New England Journal of Medicine*, **371**(11), 993–1004. doi:[10.1056/NEJMoa1409077](https://doi.org/10.1056/NEJMoa1409077).
- Mütze T, Glimm E, Schmidli H, Friede T (2019). “Group sequential designs for negative binomial outcomes.” *Statistical Methods in Medical Research*, **28**(8), 2326–2347. doi:[10.1177/0962280218773115](https://doi.org/10.1177/0962280218773115).
- National Research Council (2010). *The Prevention and Treatment of Missing Data in Clinical Trials*. National Academies Press, Washington, DC. doi:[10.17226/12955](https://doi.org/10.17226/12955).
- R Core Team (2025). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. URL <https://www.R-project.org/>.
- Rubin DB (1976). “Inference and Missing Data.” *Biometrika*, **63**(3), 581–592. doi:[10.1093/biomet/63.3.581](https://doi.org/10.1093/biomet/63.3.581).
- Venables WN, Ripley BD (2002). *Modern Applied Statistics with S*. Fourth edition. Springer, New York. ISBN 0-387-95457-0, URL <https://www.stats.ox.ac.uk/pub/MASS4/>.
- Wedzicha JA, Banerji D, Chapman KR, Vestbo J, Roche N, Ayers RT, Thach C, Fogel R, Patalano F, Vogelmeier CF (2016). “Indacaterol–glycopyrronium versus salmeterol–fluticasone for COPD.” *New England Journal of Medicine*, **374**(23), 2222–2234. doi:[10.1056/NEJMoa1516385](https://doi.org/10.1056/NEJMoa1516385).
- Wickham H (2016). *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. ISBN 978-3-319-24277-4. URL <https://ggplot2.tidyverse.org>.
- Wickham H, Hesselberth J, Salmon M, Roy O, Brueggemann S (2025a). *pkgdown: Make Static HTML Documentation for a Package*. doi:[10.32614/CRAN.package.pkgdown](https://doi.org/10.32614/CRAN.package.pkgdown). R package version 2.2.0, URL <https://CRAN.R-project.org/package=pkgdown>.
- Wickham H, Pedersen TL, Seidel D (2025b). *scales: Scale Functions for Visualization*. doi:[10.32614/CRAN.package.scales](https://doi.org/10.32614/CRAN.package.scales). R package version 1.4.0, URL <https://CRAN.R-project.org/package=scales>.
- Xie Y (2025). *knitr: A General-Purpose Package for Dynamic Report Generation in R*. R package version 1.51, URL <https://yihui.org/knitr/>.
- Zhu H, Lakkis H (2014). “Sample size calculation for comparing two negative binomial rates.” *Statistics in Medicine*, **33**(3), 376–387. doi:[10.1002/sim.5947](https://doi.org/10.1002/sim.5947).

Affiliation:

Keaven Anderson
Merck & Co., Inc., Rahway, NJ, USA
E-mail: keaven_anderson@merck.com

Hongtao Zhang
Merck & Co., Inc., Rahway, NJ, USA

Nan Xiao
Genentech, Inc., South San Francisco, CA, USA

Andrea Maes
Merck & Co., Inc., Rahway, NJ, USA